A Project Report on

"AI-Based Vehicle Detection and Counting System"

Submitted in the partial fulfillment of the requirements for the award of the Degree of

Bachelor of Computer Application

Semester - VI (2025-26)

Submitted by

230801563 – Sheladiya Brijesh H.

Guided by

Mr.Mehul Makwana

Submitted to

Faculty of Science

Department of Computer Science

Atmiya University

ATMIYA UNIVERSITY

(Established under the Gujarat Private University Act 11, 2018)

Yogidham Gurukul, Kalawad Road, Rajkot - 360005, Gujarat (INDIA)

Project ID: BCAC4270 **Date:** 04-04-2026

CERTIFICATE

This is to certify that,

Mr. / Ms. / Mrs. Sheladiya Brijesh Hasmukhbhai of

Bachelor of Computer Application

Semester VI

Has satisfactorily completed the project on

Real-Time Vehicle Detection and Counting System Using Deep Learning and Streamlit

For, Department of Computer Science,

Faculty of Science, ATMIYA University, Rajkot, Gujarat (India).

Signature
(Project Guide)

Signature
(Head of the Department)

+91 281 2563445 +91 281 2563952 admin@atmiyauni.ac.in www.atmiyauni.ac.in

DECLARATION

I hereby declare that the project report titled "**AI-Based Vehicle Detection and Counting System**"is my original work and has been completed under the guidance of Mr.Mehul Makwana . This project is submitted in the partial fulfillment of the requirements for the award of the Degree of Bachelor of Computer Application, Semester – VI (2025–26) under the Faculty of Science,Department of Computer Science, Atmiya University, Rajkot, Gujarat-India.

I also declare that this work has not been submitted to any other university or institution for any degree, diploma, or certificate.

Name of Student: Sheladiya Brijesh

Enrollment No.: 230801563

Signature:

Date:

ACKNOWLEDGEMENT

I express my deep sense of gratitude to the following respected authorities of Atmiya University for their constant support, guidance, and encouragement throughout the completion of this project.

I am sincerely thankful to **P. P. Tyagvallabh Swami** (President, Atmiya University) for providing an excellent educational environment and continuous encouragement toward academic excellence.

I extend my gratitude to **Dr.D. D. Vyas**(Registrar, Atmiya University) for ensuring smooth academic processes and administrative support that contributed to the successful completion of this project.

I am grateful to **Er. R. S. Tank**(Dean, Faculty of Science) for his valuable support, motivation, and guidance throughout the duration of the program.

My heart felt thanks to **Dr.D.P.Gohel**(Head,DepartmentofComputerScience) for providing necessary facilities, timely support, and continuous encouragement during the project work.

I would like to express my special appreciation and sincere thanks to my project guide **Mr.Mehul Makwana**, whose expert guidance, patience, and valuable suggestions helped me complete this project successfully.

I am also thankful to all the faculty members of the Department of Computer Science for their support, feedback, and academic insights throughout the course.

Finally, I gratefully acknowledge the constant support, motivation, and encouragement of my family and friends, without which this project would not have been completed.

ABSTRACT

The rapid urbanization and increasing vehicular density on Indian roads have created an urgent need for intelligent traffic monitoring and management solutions. Traditional methods of vehicle counting, such as manual enumeration and pneumatic road tubes, are labour-intensive, error-prone, and incapable of scaling to meet the demands of modern urban infrastructure. This project, titled "Vehicle Counting System Using Python," addresses these challenges by developing an automated, real-time vehicle detection and counting system that leverages state-of-the-art deep learning and computer vision technologies.

The system is built around the YOLOv8 (You Only Look Once, version 8) object detection model, which is renowned for its exceptional speed and accuracy in detecting objects within video frames. The application processes pre-recorded or live traffic video feeds, identifies individual vehicles — specifically cars and trucks — and counts them as they cross a predefined virtual detection line drawn across the road. Each detected vehicle is assigned a unique tracking identifier to prevent duplicate counting and to maintain accurate longitudinal records of traffic flow.

Table of Content

SN	Chapter	Page No.
	Certificate	I
	Declaration	II
	Acknowledgment	III
	Abstract	IV
	Table of Contents	V
1	Introduction	1
2	Problem Definition	2
3	Objectives of the Study	3
4	System Analysis	4
5	System Design	5
	5.1 Data Flow Diagrams (DFD)	
	5.2 UML Diagrams	
	5.3 ER Diagram	
6	Implementation	6
7	Testing	7
8	Output Screenshots	8
9	Conclusion	9
10	Future Scope	10
11	Bibliography / References	11
12	Appendix	12

INTODUCTION

Urbanization has led to a massive increase in vehicle density, making manual traffic management nearly impossible. Traditional surveillance systems act merely as "passive observers," recording footage that is rarely analyzed unless an incident occurs. The **Vision AI-Based Vehicle Detection and Counting System** transforms these passive feeds into "active data streams".

This project focuses on using deep learning models to identify objects within video frames, specifically focusing on vehicles such as cars, trucks, and motorcycles. The system is designed to provide real-time feedback to administrators, allowing for better traffic flow optimization and security monitoring. As part of the **Bachelor of Computer Application** curriculum, this project demonstrates the practical application of computer science principles in solving real-world transportation challenges.

PROBLEM DEFINITION

Traffic congestion is one of the most pressing challenges facing urban areas in India and around the world. According to a report by the Ministry of Road Transport and Highways, Government of India, the total number of registered motor vehicles in the country exceeded 326 million in the year 2022, with an annual growth rate of approximately 9 to 10 per cent. This explosive growth in the vehicle population has placed immense strain on the existing road infrastructure, leading to chronic congestion, increased travel times, elevated levels of air pollution, and a higher frequency of road accidents.

Effective traffic management requires accurate and timely data about the volume and composition of traffic on different road segments. Traffic volume data — specifically the number and types of vehicles passing through a given point over a defined period — is one of the most fundamental inputs for transportation planning, traffic signal optimization, road maintenance scheduling, and infrastructure investment decisions. Without reliable traffic count data, transportation authorities are forced to rely on estimates and assumptions, which can lead to suboptimal resource allocation and policy decisions.

OBJECTIVES OF THE STUDY

The following objectives were defined at the outset of this project to guide its design, development, and evaluation:

To develop an automated vehicle detection and counting system using deep learning. The primary objective of this project is to build a system that can automatically detect and count vehicles in traffic video footage without requiring manual intervention. The system employs the YOLOv8 object detection model, which has been pre-trained on the COCO dataset containing 80 object classes, including cars and trucks. By leveraging transfer learning, the system can accurately detect these vehicle categories in diverse traffic scenarios without the need for custom model training.

To implement real-time object tracking for accurate vehicle counting. A key challenge in video-based vehicle counting is ensuring that each vehicle is counted exactly once, regardless of how many frames it appears in. To address this, the project implements the built-in object tracking capabilities of YOLOv8, which assigns a unique tracking identifier to each detected vehicle and maintains this identifier across consecutive frames. Vehicles are counted only when they cross a predefined virtual detection line, preventing duplicate counts.

To build an interactive web-based dashboard for traffic analysis. The project aims to provide a user-friendly interface that allows non-technical users to upload video files, configure analysis parameters, and view results without requiring any programming knowledge or command-line interaction. The Streamlit framework is used to build this dashboard, which features real-time metric updates, animated visualizations, and data export capabilities.

SYSTEM ANALYSIS

4.1 EXISTING SYSTEM

WORKING METHOD

In the manual counting approach, trained enumerators are stationed at strategic locations along roads and intersections. They use tally counters or paper-based forms to record each vehicle that passes their observation point. The data is typically aggregated into hourly and daily totals, which are then compiled into traffic volume reports.

LIMITATIONS

Manual counting is labour-intensive, time-consuming, and prone to human error, particularly during periods of high traffic volume or adverse weather conditions.

Manual counting cannot provide real-time data, as the collected data must be compiled and processed after the counting exercise is completed.

RISKS AND INEFFICIENCIES

The reliance on outdated vehicle counting methods introduces several risks and inefficiencies into the traffic management process. Inaccurate vehicle count data can lead to flawed traffic signal timing, resulting in unnecessary delays and increased fuel consumption.

4.2 PROPOSED SYSTEM

SYSTEM WORKFLOW

Environment Detection: Upon upload, the system extracts the first frame of the video and performs brightness analysis using the HSV colour space to classify the environment as Day or Night. This classification is displayed prominently on the dashboard.

Parameter Configuration: The user may optionally configure the vehicle count limit, which specifies the maximum number of vehicles to count before stopping the analysis.

Analysis Initiation: The user clicks the “Start Analysis” button to begin processing. The system loads the YOLOv8 nano model (cached for subsequent runs) and begins processing the video frame by frame.

4.3 FEASIBILITY STUDY

TECHNICAL FEASIBILITY

The technical feasibility of the proposed system is well-established. All the core technologies used in the system — YOLOv8, OpenCV, Streamlit, Pandas, Plotly, and Python — are mature, well-documented, open-source technologies with large and active community support.

ECONOMIC FEASIBILITY

The economic feasibility of the proposed system is highly favourable. All software components used in the project are open-source and available at no cost, eliminating software licensing expenses.

OPERATIONAL FEASIBILITY

The operational feasibility of the system is supported by its intuitive web-based interface, which requires minimal technical training to use. The Streamlit dashboard presents a clean, straightforward workflow: upload a video, optionally configure parameters, and click a button to start analysis.

4.4 REQUIREMENT SPECIFICATION

4.4.1 HARDWARE REQUIREMENTS

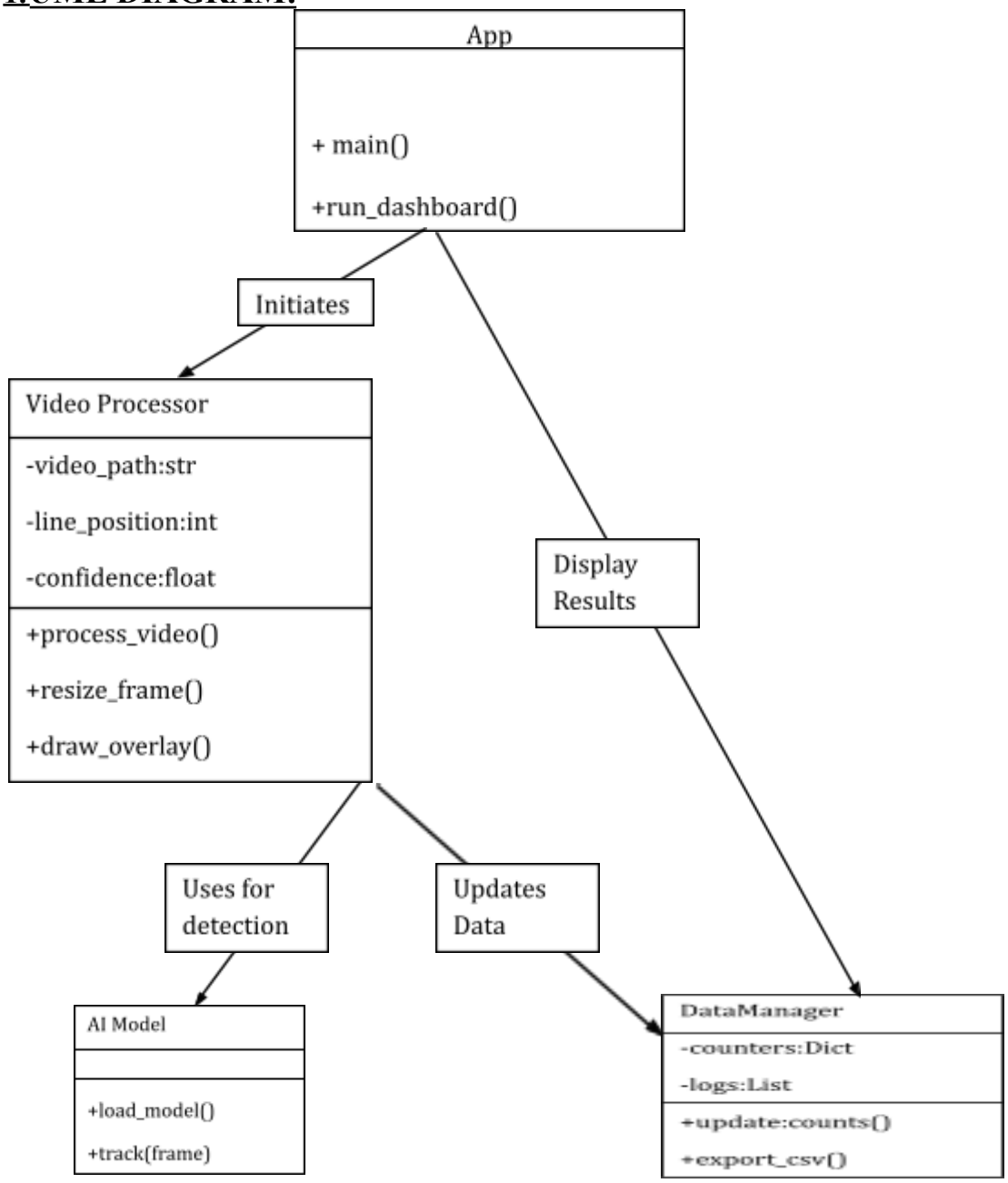
Component	Minimum Specification	Recommended Specification
Processor	Intel Core i3 or equivalent	Intel Core i5 / AMD Ryzen 5 or higher
RAM	4 GB	8 GB or more
Storage	500 MB free disk space	2 GB or more for video storage
Display	1366 x 768 resolution	1920 x 1080 resolution
GPU (Optional)	Not required	NVIDIA GPU with CUDA support
Network	Not required for local operation	Internet for initial package download

4.4.2 SOFTWARE REQUIREMENTS

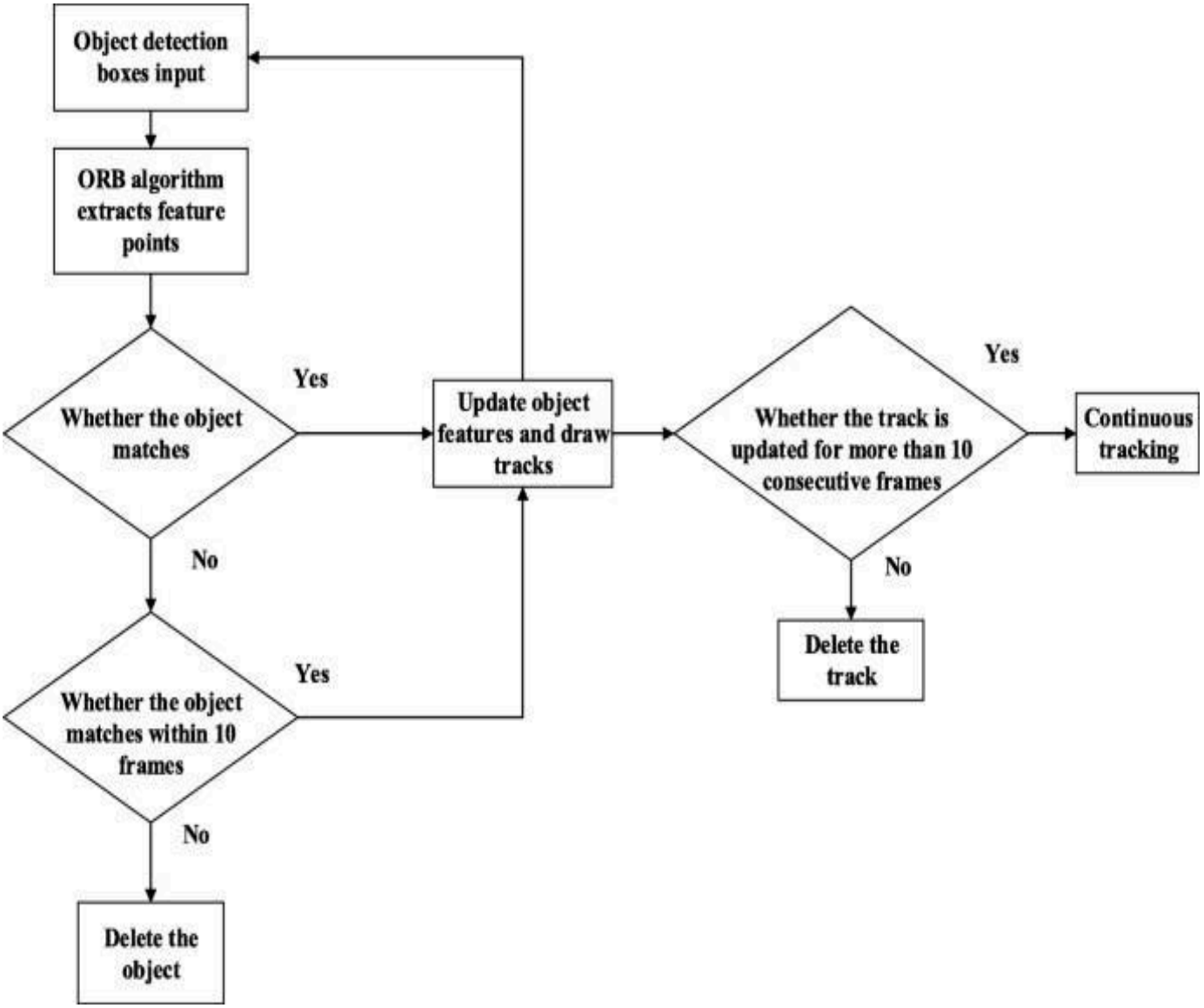
Component	Specification
Operating System	Windows 10/11, macOS, or Ubuntu 20.04+
Programming Language	Python 3.9 or higher
Web Framework	Streamlit
Object Detection Model	YOLOv8 Nano (Ultralytics)
Computer Vision Library	OpenCV (opencv-python-headless)
Data Processing	Pandas, NumPy
Visualization	Plotly, Matplotlib

SYSTEM DESIGN

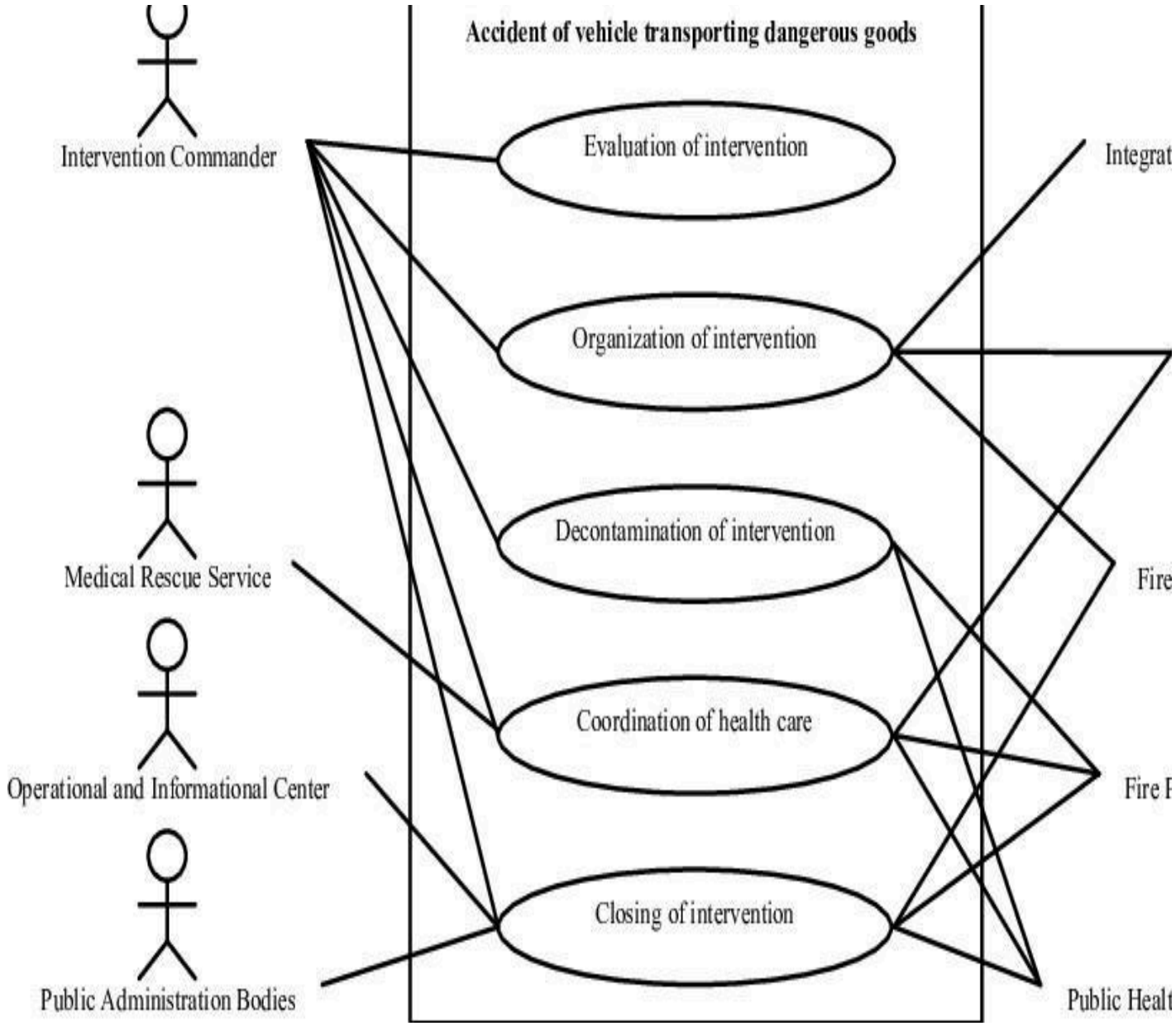
1.UML DIAGRAM:



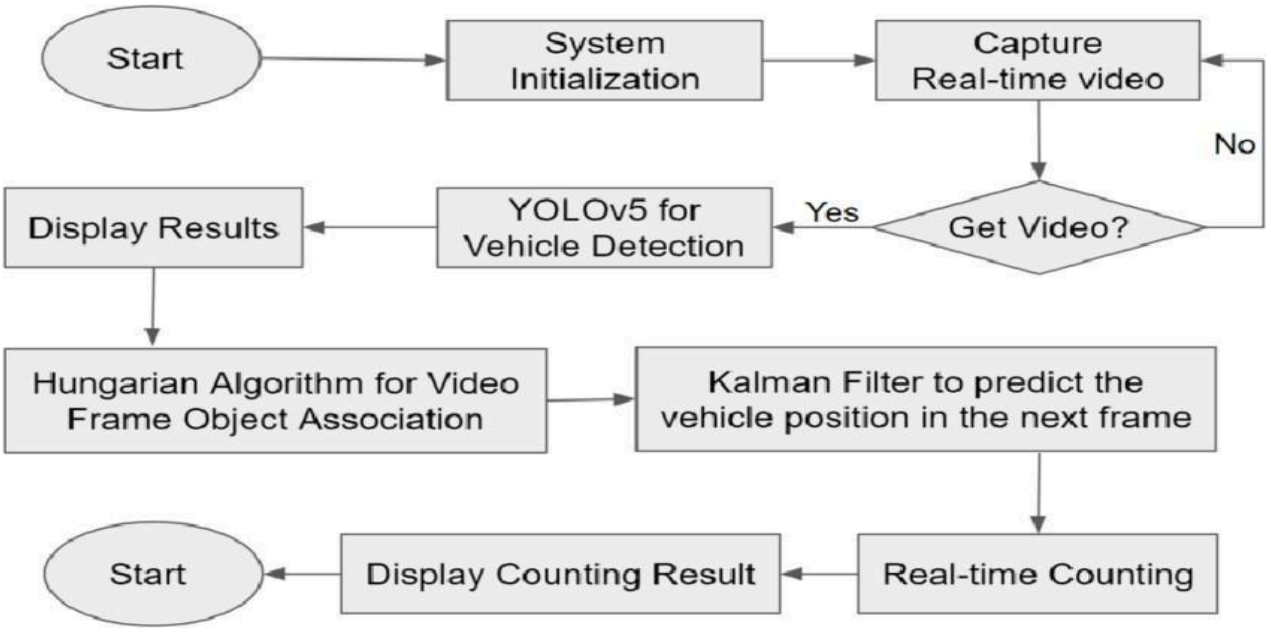
2.ER DIAGRAM [ENTITY RELATIONSHIP]:



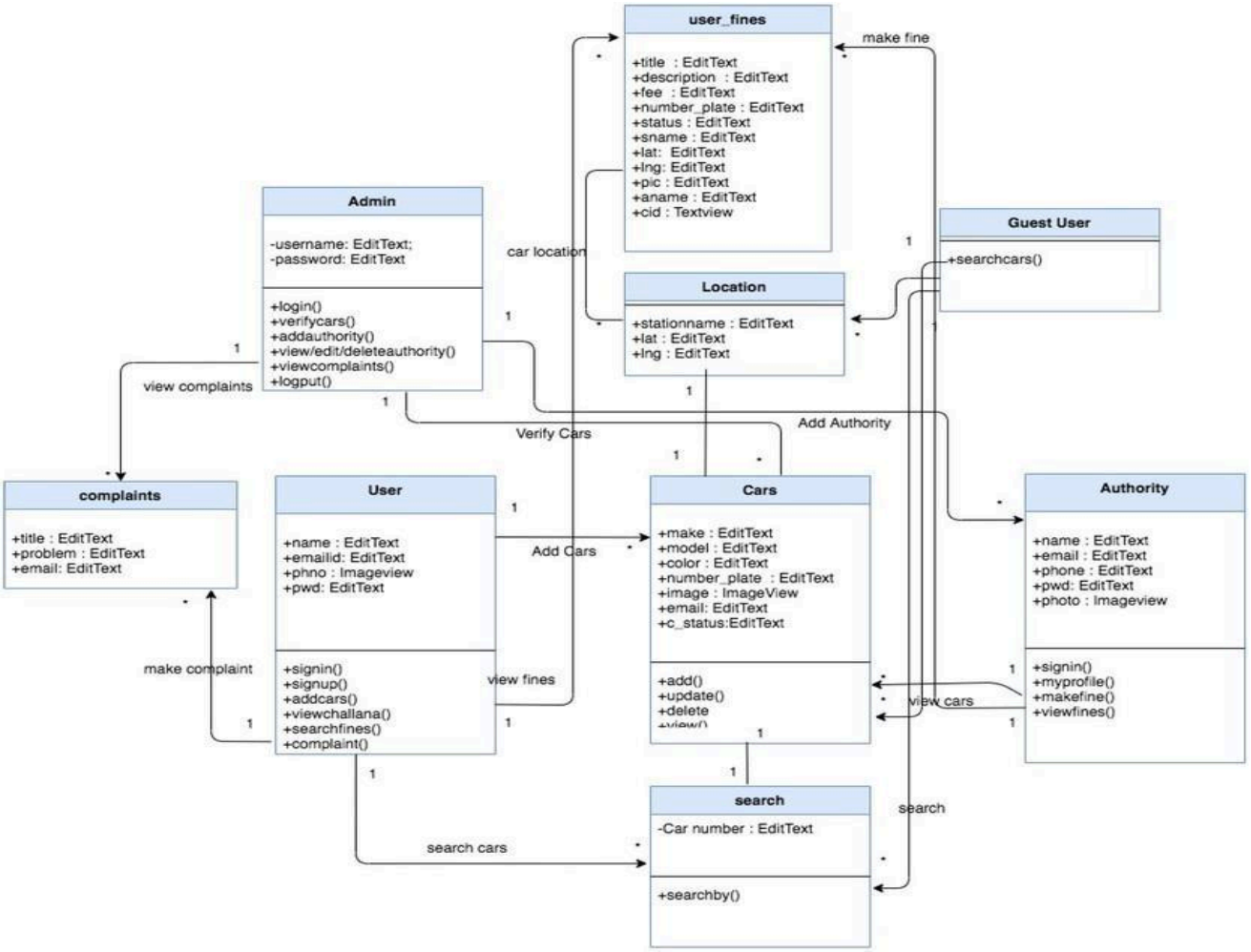
3.USE CASE DIAGRAM:



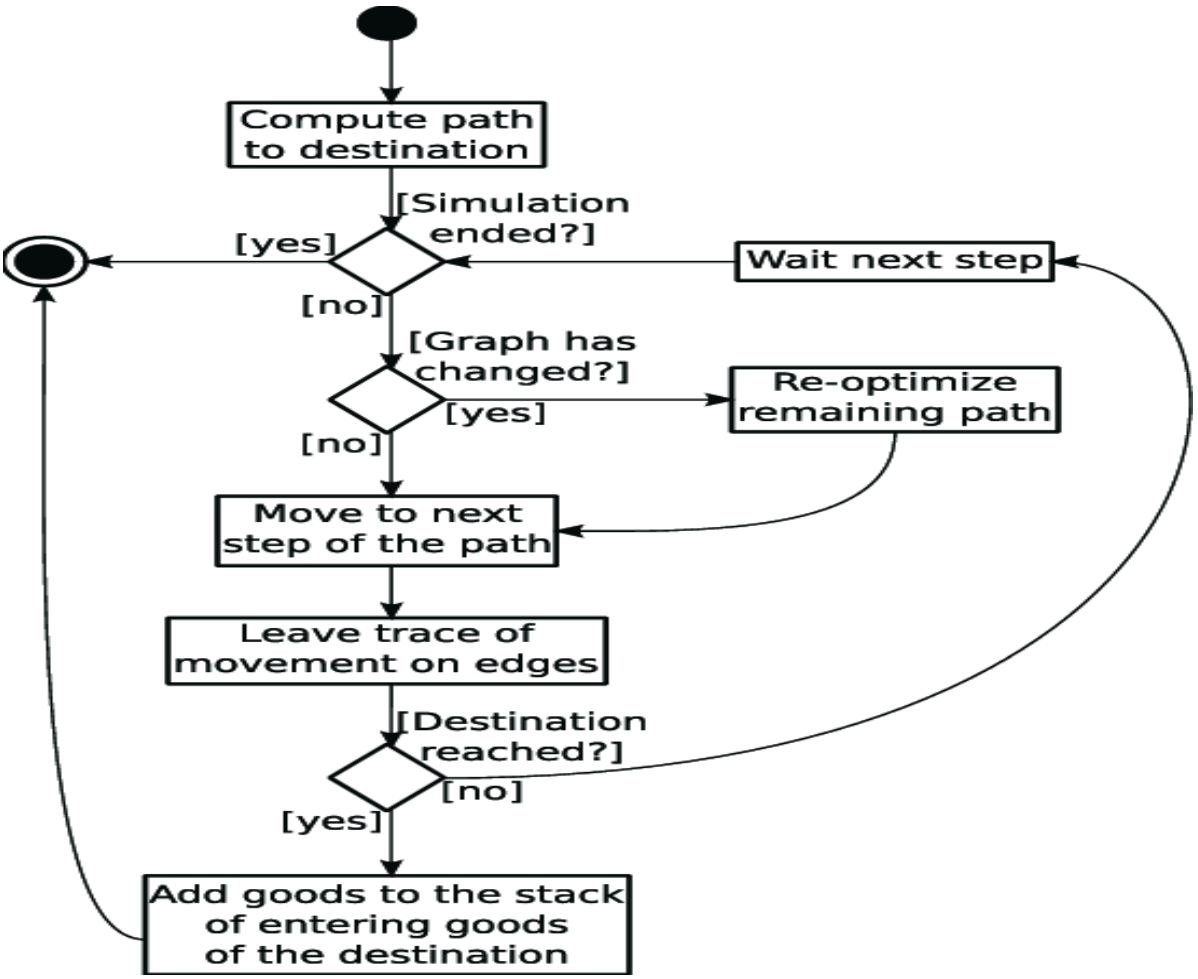
4. DATA FLOW DIAGRAM:



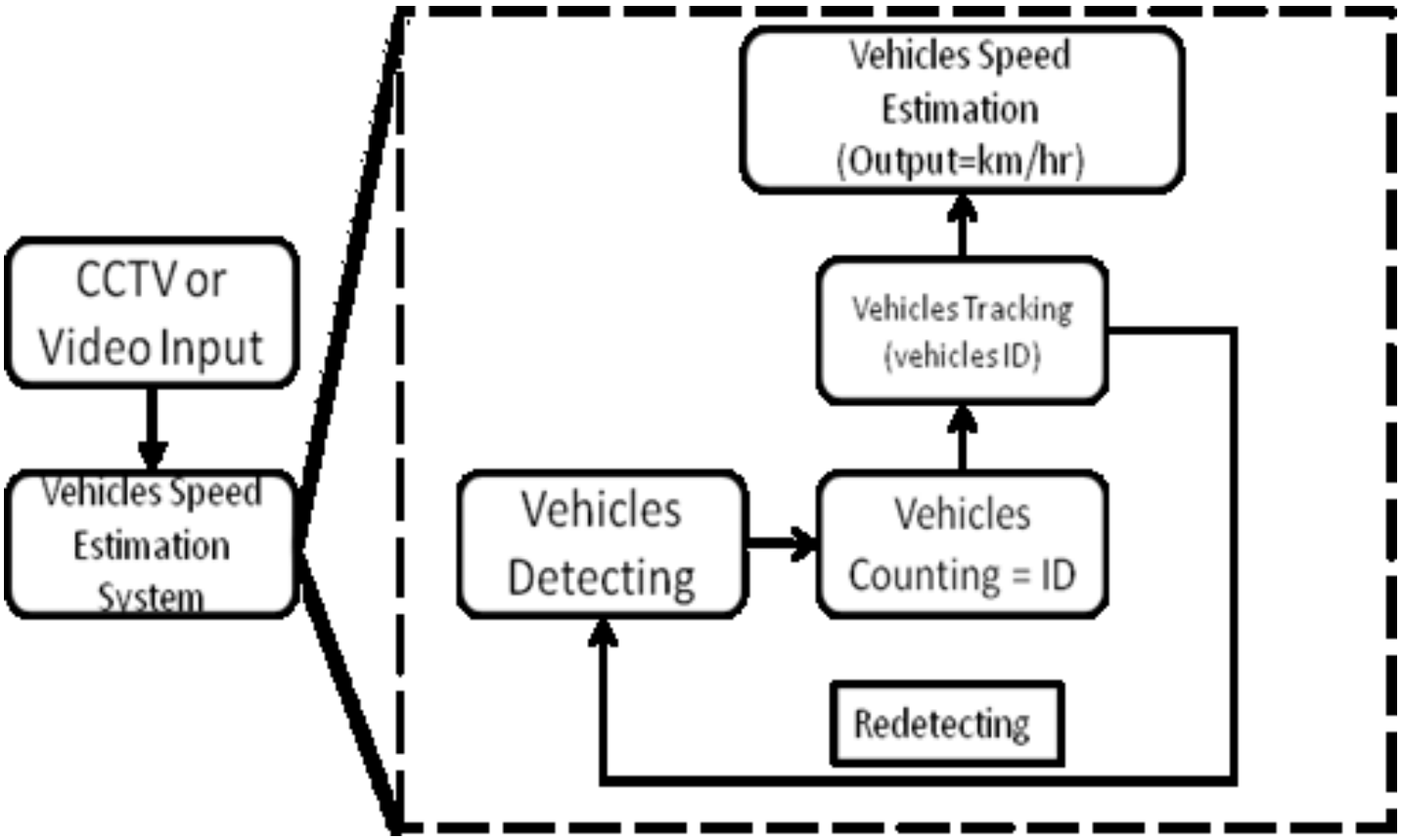
5. CLASSDIAGRAM:



6.ACTIVITY DIAGRAM



7.SYSTEM ARCHITECTURE



IMPLEMENTATION

MODULE-WISE EXPLANATION

The Vehicle Counting System is organized into several logical modules, each responsible for a distinct aspect of the system's functionality. The modular design promotes code maintainability, readability, and extensibility

MODEL LOADING MODULE

Purpose: To load and cache the YOLOv8 object detection model for efficient reuse across analysis sessions.

Input: The path to the pre-trained YOLOv8 nano model weights file (yolov8n.pt).

Process: The module utilizes Streamlit's @st.cache_resource decorator to load the YOLO model once and cache it in memory. On subsequent invocations, the cached model instance is returned directly, avoiding redundant model loading and significantly reducing startup time for repeat analyses.

Output: An initialized YOLO model object ready for inference and tracking operations.

ENVIRONMENT DETECTION MODULE

Purpose: To classify the lighting environment of the video scene as Day or Night based on frame brightness analysis.

Input: A single video frame in BGR colour format, as read by OpenCV.

Process: The module converts the input frame from BGR to HSV (Hue, Saturation, Value) colour space using OpenCV. It then calculates the mean value of the V (Value) channel, which represents the overall

DATA EXPORT MODULE

PURPOSE: To compile detection records into a structured format and provide downloadable data exports.

INPUT: The detection log list containing dictionaries with Timestamp, Vehicle ID, and Type fields.

PROCESS: The module converts the detection log into a Pandas DataFrame. Vehicle IDs are cast to string type to ensure uniform data typing. A summary row is appended to the DataFrame containing aggregate statistics. The DataFrame is encoded as a CSV string in UTF-8 format and made available for download through Streamlit's download button widget.

OUTPUT: An interactive data table displayed on the dashboard and a downloadable CSV file.

TECHNOLOGIES USED

PYTHON (VERSION 3.9+)

Python is a high-level, interpreted, general-purpose programming language known for its clean syntax, extensive standard library, and vast ecosystem of third-party packages. readability or maintainability.

YOLOV8 (ULTRALYTICS)

YOLOv8, developed by Ultralytics, is a state-of-the-art object detection and tracking framework.

OPENCV (OPEN SOURCE COMPUTER VISION LIBRARY)

OpenCV is the most widely used open-source computer vision library, providing over 2,500 algorithms for image and video processing, feature detection, object recognition, and more.

STREAMLIT

Streamlit is an open-source Python framework for building interactive web applications with minimal code.

PANDAS

Pandas is a powerful data manipulation and analysis library for Python, built on top of NumPy.

PLOTLY

Plotly is an interactive data visualization library that generates publication-quality charts and graphs.

NUMPY

NumPy is the foundational package for numerical computing in Python.

MATPLOTLIB

Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python.

TESTING

TYPES OF TESTING PERFORMED

The Vehicle Counting System was subjected to a comprehensive testing regimen to ensure the correctness, reliability, and usability of all system components. The following types of testing were performed:

UNIT TESTING

Unit testing was conducted on individual functions and modules of the system in isolation. Each function — including the model loading function, the environment detection function, the video processing function, and the data export .

INTEGRATION TESTING

Integration testing was performed to verify that the individual modules work correctly when combined. This involved testing the interaction between the video processing module and the YOLOv8 detection engine, the flow of detection data from the counting logic to the data logger, and the rendering of visualization components from the processed data.

SYSTEM TESTING

System testing evaluated the complete application from end to end, simulating a typical user workflow. This involved uploading test video files through the Streamlit dashboard, initiating analysis, observing real-time metric updates, verifying the accuracy of final counts.

ACCEPTANCE TESTING

Acceptance testing determined whether the system satisfies the project's defined objectives and requirements. This involved reviewing the system's functionality against the stated objectives, verifying the accuracy of vehicle counts .

SYSTEM TESTING (FUNCTIONAL)

The complete application flow was tested from the user interface.

Test Case ID	**Test Case Description**	**Input Data**	**Expected Outcome**
TC-01	Video Upload	Valid .mp4 file (10MB)	File uploads successfully; preview image appears; "Start Analysis" button becomes active.
TC-02	Invalid File Upload	.txt or .jpg file	System should reject the file or not allow selection.
TC-03	Day/Night Detection	Video recorded at night	System identifies "Night" mode and displays the dark blue badge.
TC-04	Vehicle Countin g	Videowith 3 cars crossing the line	Counterfor"Cars"increasestoexactly 3.
TC-05	LinePosition Adjustment	Change Slider value to 300	Thereddetectionlinemovesupinthe preview and processing video.
TC-06	Stop Limit	Set"Stopafter X Vehicles" to 5	Analysis stops automatically once total count reaches 5.
TC-07	Report Generation	Complete analysis	"DownloadReport"buttonappears;CSV con

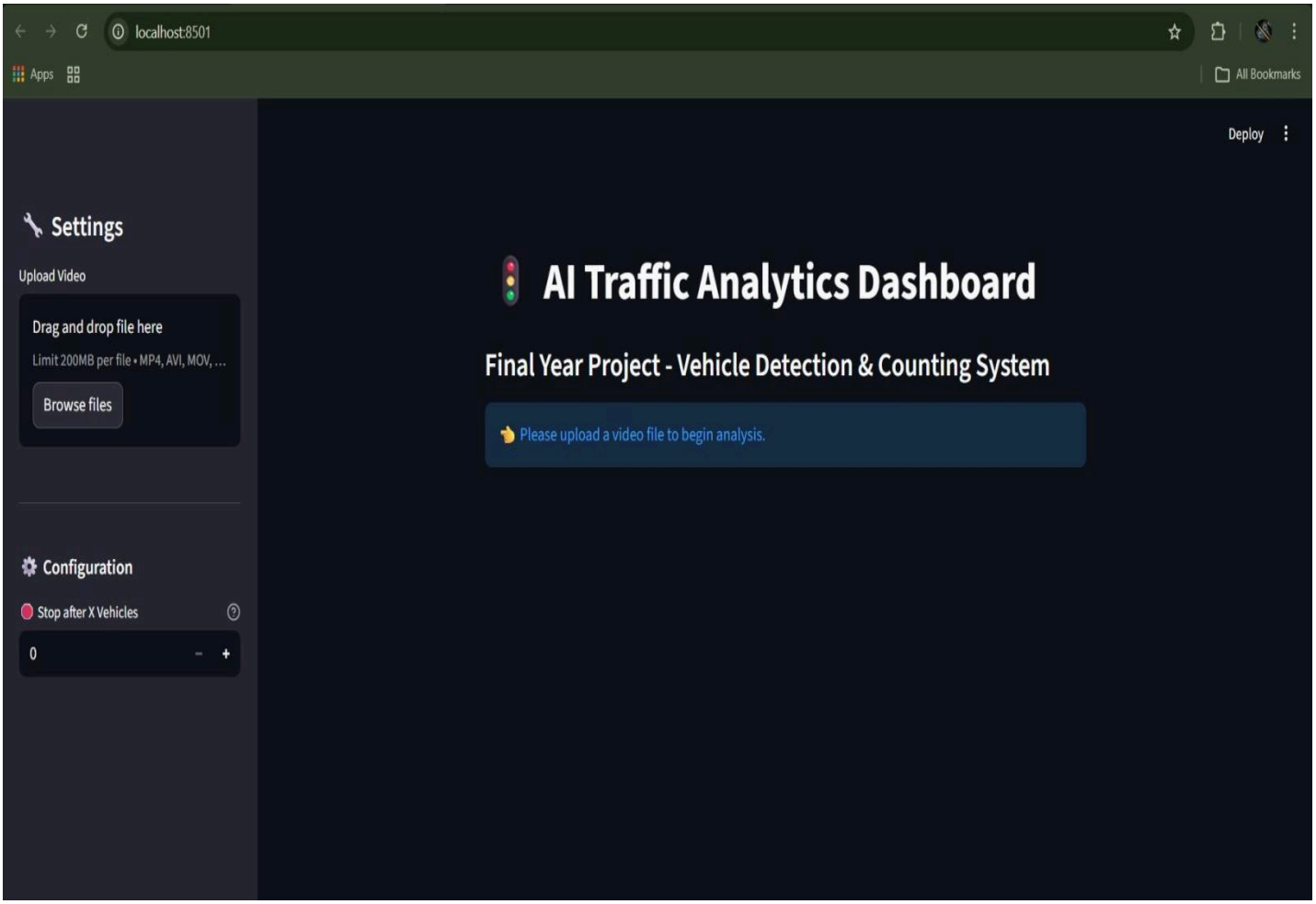
EXPECTED VS ACTUAL RESULTS

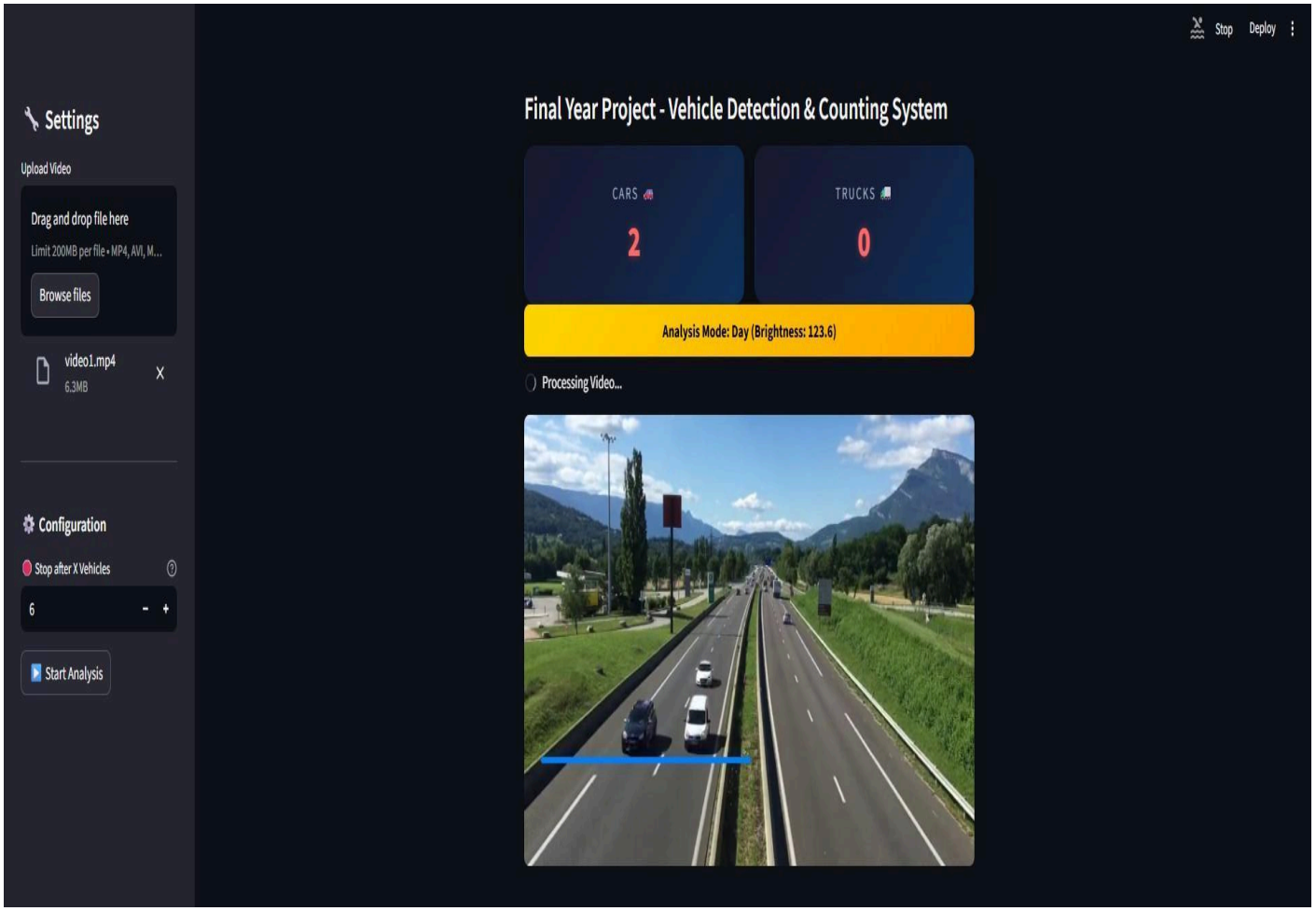
The testing results demonstrated a high level of consistency between expected and actual outcomes across all test cases. The vehicle detection module reliably identified cars and trucks in video frames under both daytime and nighttime conditions, though detection accuracy was observed to decrease marginally in very low-light or heavily occluded scenarios. The tracking mechanism effectively prevented duplicate counting, with each vehicle being assigned a unique and persistent tracking identifier across frames.

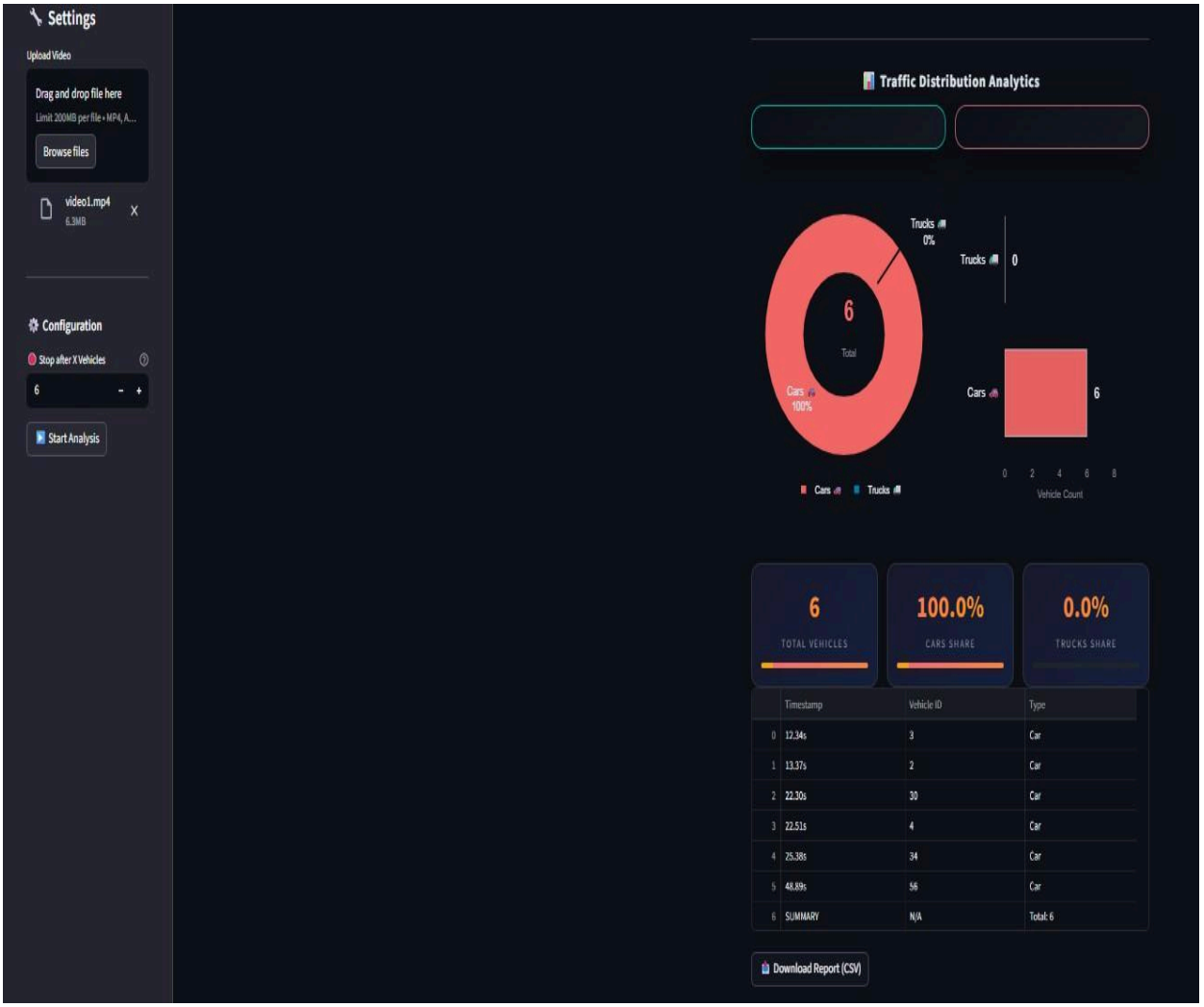
The environment detection module correctly classified lighting conditions in the vast majority of test scenarios. A minor observation was that transitional lighting conditions during dawn or dusk could occasionally result in fluctuating classifications, though this did not impact the accuracy of vehicle counting.

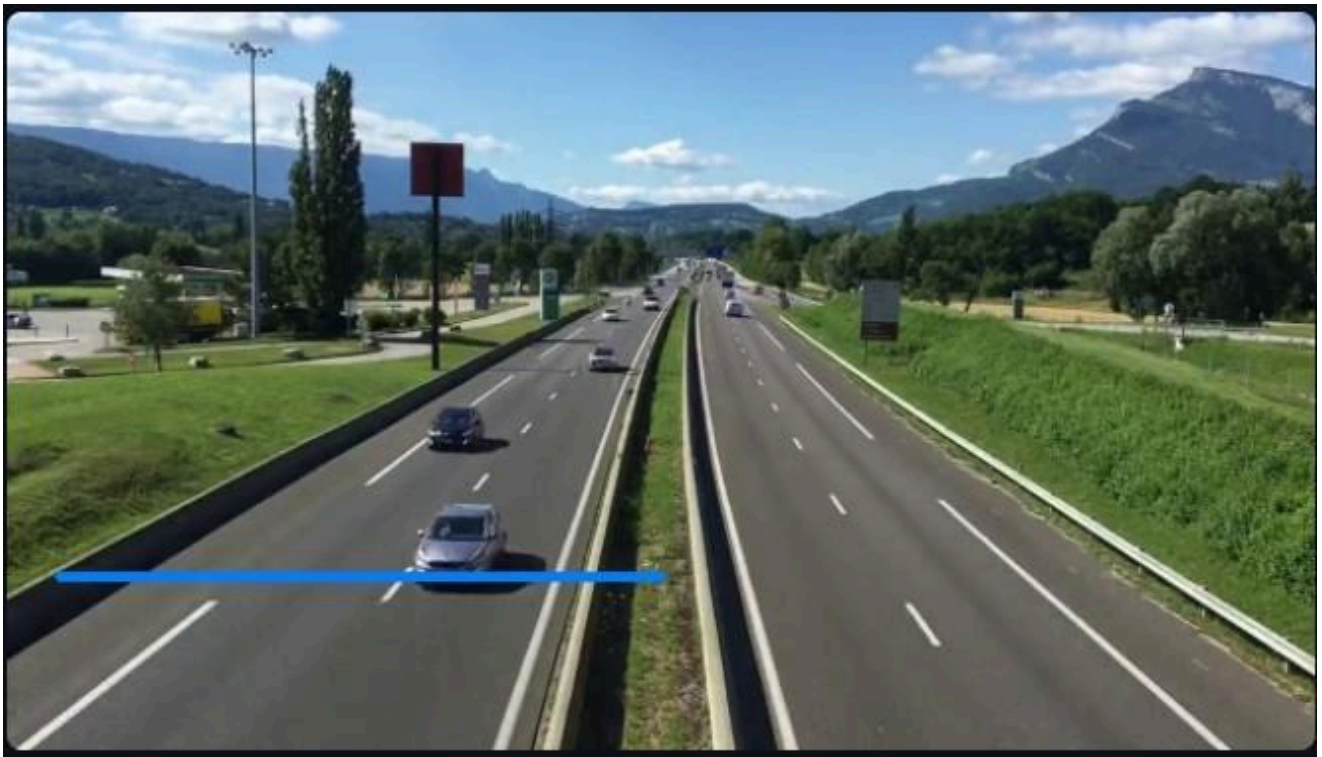
The vehicle count limit feature performed as expected, reliably halting the analysis once the specified count threshold was reached. The CSV export consistently produced correctly formatted files with accurate and complete detection records, including the summary row with aggregate statistics.

OUTPUT SCREENSHOTS




Stop
Deploy
Settings
Upload Video
Drag and drop file here
Limit 200MB per file • MP4, AVI, M...
Browse files
video1.mp4
6.3MB
X
Configuration
Stop after X Vehicles
6
-
+
Start Analysis
Final Year Project - Vehicle Detection & Counting System
CARS
2
TRUCKS
0
Analysis Mode: Day (Brightness: 123.6)
Processing Video...


Settings
Upload Video
Drag and drop file here
Limit 200MB per file • MP4, A...
Browse files
video1.mp4
6.3MB
X
Configuration
Stop after X Vehicles
6
Start Analysis
Traffic Distribution Analytics
Trucks
0%
6
Total
Cars
100%
Cars
Trucks
Trucks
0
Cars
6
0 2 4 6 8
Vehicle Count
6
TOTAL VEHICLES
100.0%
CARS SHARE
0.0%
TRUCKS SHARE
Timestamp
Vehicle ID
Type
0 12.34s 3 Car
1 13.37s 2 Car
2 22.30s 30 Car
3 22.51s 4 Car
4 25.38s 34 Car
5 48.89s 56 Car
6 SUMMARY N/A Total: 6
Download Report (CSV)



Analysis Complete! 🎉

Total Cars

6

Total Trucks

0

Grand Total

6

CONCLUSION

The “Vehicle Counting System Using Python” project has been successfully designed, developed, and tested as a comprehensive, automated traffic monitoring solution that leverages modern artificial intelligence and computer vision technologies to detect, classify, and count vehicles in traffic video footage. The project demonstrates the practical applicability of deep learning-based object detection in the domain of intelligent transportation systems and represents a meaningful contribution to the field of automated traffic analysis.

The primary objective of the project — to develop an automated system that can count vehicles without human intervention — has been fully achieved. The system employs the YOLOv8 nano model for object detection and tracking, OpenCV for video processing, and a suite of Python data science libraries for analytics and visualization, all integrated into an interactive and visually appealing web-based dashboard built with Streamlit. The system accurately detects and classifies two categories of vehicles — cars and trucks — and counts them as they cross a virtual detection line, with tracking-based deduplication ensuring that each vehicle is counted exactly once.

The interactive dashboard provides a seamless user experience, from video upload and parameter configuration to real-time analysis monitoring and post-analysis exploration. The animated metric cards, interactive Plotly charts, and summary statistics cards transform raw detection data into actionable insights that are immediately interpretable by both technical and non-technical users. The CSV export feature ensures that the detection data can be preserved and analysed further using external tools, supporting integration into broader data analysis and reporting workflows.

FUTURE SCOPE

The Vehicle Counting System, in its current form, provides a solid foundation for automated traffic monitoring. Several enhancements and extensions are envisioned to expand its capabilities and improve its applicability in real-world traffic management scenarios:

SUPPORT FOR ADDITIONAL VEHICLE CATEGORIES: The system can be extended to detect and classify additional vehicle types, including motorcycles, buses, auto-rickshaws, bicycles, and emergency vehicles. This would require either fine-tuning the YOLOv8 model on a custom dataset containing these additional classes or integrating a larger pre-trained model that supports a broader set of vehicle categories.

REAL-TIME LIVE CAMERA STREAM PROCESSING: Integrating support for live video streams from IP cameras using RTSP (Real-Time Streaming Protocol) or HTTP streaming protocols would enable the system to function as a real-time traffic monitoring tool deployed at intersections and roadways. This would require modifications to the video processing pipeline to handle continuous stream input rather than finite video files.

VEHICLE SPEED ESTIMATION: By tracking the pixel displacement of vehicles across consecutive frames and calibrating the camera's perspective transformation, the system can estimate the speed of passing vehicles. This capability would add significant value for speed enforcement, congestion monitoring, and road safety analysis.

DIRECTION AND LANE-WISE COUNTING: The system can be enhanced to detect the direction of vehicle movement (e.g., northbound vs. southbound) and to count vehicles independently for each traffic lane. This granularity of data is essential for traffic signal optimization, lane utilization analysis, and directional traffic flow studies.

CLOUD DEPLOYMENT AND MULTI-CAMERA SUPPORT: Deploying the system on cloud infrastructure (such as AWS, Google Cloud, or Microsoft Azure) would enable processing of video feeds from multiple cameras simultaneously, supporting city-wide traffic monitoring. A centralized web dashboard could aggregate and display data from all connected cameras on a single interface.

BIBLIOGRAPHY/REFERENCES

J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 779-788.

G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLOv8,” Ultralytics, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics. [Accessed: Feb. 2026].

G. Bradski, “The OpenCV Library,” *Dr. Dobb’s Journal of Software Tools*, vol. 25, no. 11, pp. 120-125, 2000.

Streamlit Inc., “Streamlit: The Fastest Way to Build Data Apps,” 2024. [Online]. Available: https://streamlit.io/. [Accessed: Feb. 2026].

W. McKinney, “Data Structures for Statistical Computing in Python,” in *Proceedings of the 9th Python in Science Conference (SciPy 2010)*, Austin, TX, USA, 2010, pp. 56-61.

Plotly Technologies Inc., “Plotly: Modern Analytic Apps for the Enterprise,” 2024. [Online]. Available: https://plotly.com/. [Accessed: Feb. 2026].

C. R. Harris *et al.*, “Array Programming with NumPy,” *Nature*, vol. 585, no. 7825, pp. 357-362, Sep. 2020.

A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, “Simple Online and Realtime Tracking,” in *Proceedings of the IEEE International Conference on Image Processing (ICIP)*, Phoenix, AZ, USA, 2016, pp. 3464-3468.

T.-Y. Lin *et al.*, “Microsoft COCO: Common Objects in Context,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, Zurich, Switzerland, 2014, pp. 740-755.

Ministry of Road Transport and Highways, Government of India, “Annual Report 2022-23,” New Delhi, India, 2023. [Online]. Available: https://morth.nic.in/. [Accessed: Feb. 2026].

APPENDIX

APPENDIX A: PROJECT DIRECTORY STRUCTURE

```
vehicle_counter/
|-- app.py              # Main application file (Streamlit + YOLOv8)
|-- requirements.txt      # Python package dependencies
|-- run_app.ps1           # PowerShell launcher script
|-- README.txt            # Setup and usage instructions
|-- yolov8n.pt            # Pre-trained YOLOv8 Nano model weights
|-- video.mp4             # Sample traffic video file (primary)
|-- video1.mp4            # Sample traffic video file (secondary)
|-- libs/               # Pre-packaged libraries (legacy, disabled)
|-- venv/                                                    # Python virtual
environment
```

APPENDIX B: PYTHON PACKAGE DEPENDENCIES

Package	Version	Purpose
streamlit	Latest	Web dashboard framework
pandas	Latest	Data manipulation and CSV export
opencv-python-headless	Latest	Video capture and image processing
ultralytics	Latest	YOLOv8 object detection model
matplotlib	Latest	Static data visualization
plotly	Latest	Interactive charts and graphs
numpy	Latest	Numerical computation (installed as dependency)

APPENDIX C: APPLICATION LAUNCH INSTRUCTIONS

METHOD 1: USING POWERSHELL SCRIPT (RECOMMENDED)

./run_app.ps1

METHOD 2: USING COMMAND PROMPT

set PYTHONPATH = python -m streamlit run app.py

The application will launch and open in the default web browser at http://localhost:8501.

APPENDIX D: YOLOV8 COCO CLASS IDS USED

Class ID	Class Name	Used in System
2	Car	Yes
7	Truck	Yes

The YOLOv8 model is pre-trained on the COCO (Common Objects in Context) dataset, which defines 80 object categories. This project utilizes only class IDs 2 (Car) and 7 (Truck) for vehicle detection and counting.

APPENDIX E: DETECTION LINE CONFIGURATION

The virtual detection line is positioned at a fixed vertical coordinate (y = 550 pixels in the original frame resolution). The detection offset is set to 10 pixels, creating a detection zone of 20 pixels around the line centre. Vehicles are counted only when their centre point falls within this detection zone and when they are located in the left half of the frame.

APPENDIX F: SAMPLE CSV OUTPUT FORMAT

Timestamp	Vehicle ID	Type
2.34s	5	Car
3.71s	8	Truck
5.12s	12	Car
7.89s	15	Car
9.45s	19	Truck
SUMMARY	N/A	Total: 5